PROGRAMAÇÃO ORIENTADA A OBJETOS

IMPLEMENTAÇÃO E VALIDAÇÃO DO SISTEMA FITLIFE

Atividade 9 - Unidade 4

Instituição	Anhanguera
Curso	Ciência da Computação - 6º período noturno
Disciplina	Programação Orientada a Objetos
Professor	Cleidson Daniel
Aluno	Kayky Moreira Assunção
Matrícula	406392088938
Data	21 de outubro de 2026

Integrantes do grupo

Lucas Borges Marra • João Pedro Rodrigues Silva • Mateus Bonifácio Nolasco • João Pedro Rodrigues Venâncio • Kayky Moreira Assunção

1. Objetivo e cenário

A atividade propõe implementar e testar uma parte funcional da academia FitLife. A solução deve gerenciar clientes, planos, treinos e pagamentos, aplicando os conceitos trabalhados nas quatro unidades da disciplina e garantindo regras de negócio, validação de dados e tratamento de erros.

Resultado entregue

Foi desenvolvido um projeto Java 17 organizado em camadas de domínio, serviços e exceções. O módulo cadastra clientes e planos, associa o plano, registra treinos e exercícios, processa pagamentos, apresenta os dados no console e executa testes automatizados com JUnit 5.

Requisitos funcionais

- Cadastrar clientes com nome, e-mail e idade.
- Cadastrar os planos Mensal e Premium.
- Associar um plano ativo a um cliente.
- Cadastrar treinos e exercícios para o cliente.
- Calcular e processar o pagamento da mensalidade.
- Validar dados e tratar exceções com mensagens claras.
- Exibir a demonstração do sistema no console.

Organização do projeto

Pasta	Responsabilidade
src/main/java/.../model	Entidades, classe abstrata, interface e objetos de valor.
src/main/java/.../service	Regras de cadastro, associação e pagamento.
src/main/java/.../exception	Exceções específicas do domínio.
src/test/java	Testes automatizados com JUnit 5.
docs e evidencias	Diagrama, explicação e resultados da execução.

2. Conceitos de POO aplicados

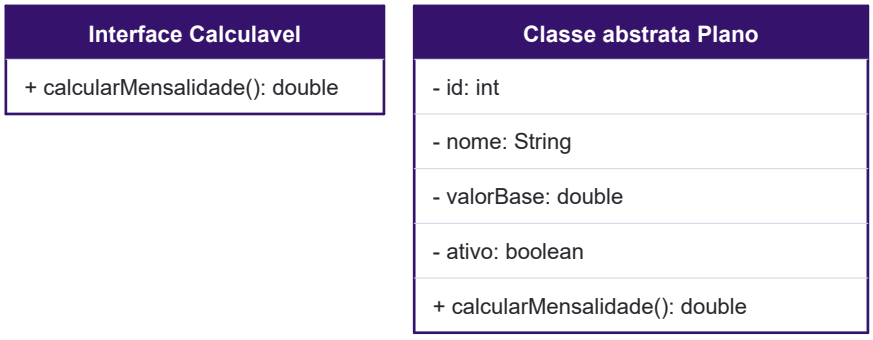
Conceito	Aplicação no FitLife
Classes e objetos	Cliente, Plano, Treino, Exercicio e Pagamento representam elementos reais da academia.
Encapsulamento	Todos os atributos são privados. Getters, setters e métodos de negócio protegem o estado dos objetos.
Herança	PlanoMensal e PlanoPremium herdam atributos e comportamentos comuns da classe abstrata Plano.
Interface	Calculavel define o contrato calcularMensalidade(), implementado pela hierarquia de planos.
Polimorfismo	Cada plano calcula o valor de modo diferente, mas o sistema trata ambos pelo tipo Plano ou Calculavel.
Associação	Cliente mantém uma referência ao Plano escolhido.
Agregação	AcademiaService reúne clientes e planos, que continuam sendo objetos independentes.
Composição	Cliente contém treinos e Treino contém exercícios, formando partes do cadastro.
Dependência	PagamentoService usa Cliente e Plano apenas durante a operação de pagamento.
Exceções	DadosInvalidosException e PagamentoException impedem estados incorretos e comunicam falhas.

Exemplo de polimorfismo

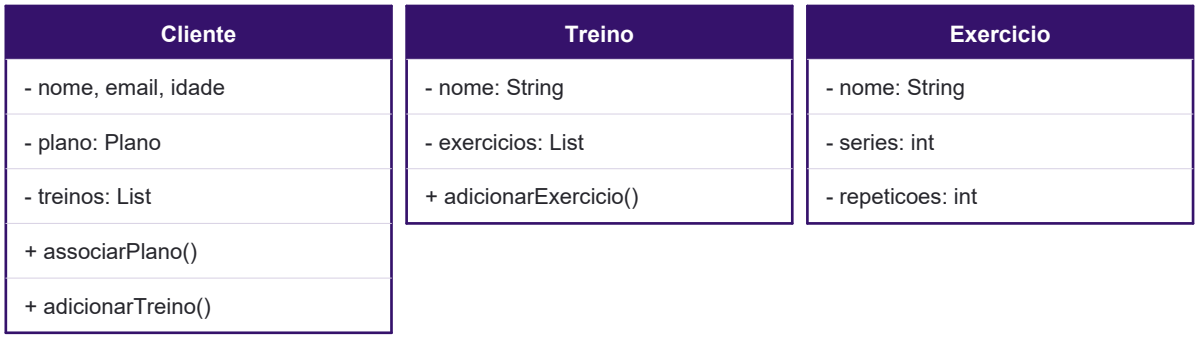
PagamentoService não precisa descobrir se o cliente possui PlanoMensal ou PlanoPremium. Ele chama plano.calcularMensalidade(), e o objeto concreto executa sua própria regra: R$ 100,00 no plano Mensal e R$ 250,00 no Premium.

3. Diagrama de classes

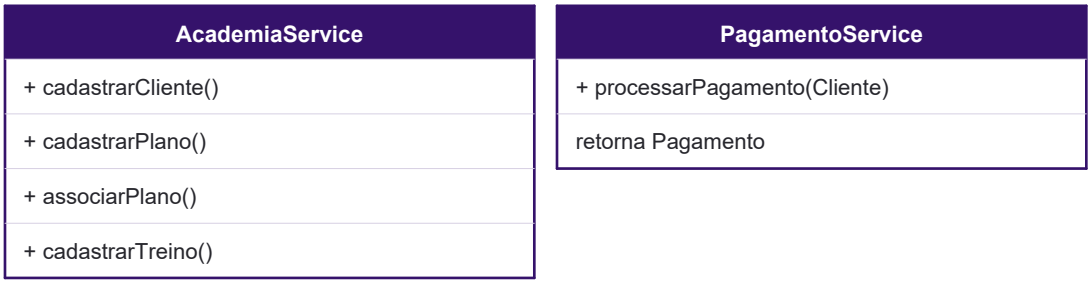
O diagrama abaixo resume as classes principais, seus atributos, métodos e relacionamentos. A versão editável em Mermaid está disponível em docs/DIAGRAMA_CLASSES.md.



Plano implementa Calculavel | PlanoMensal e PlanoPremium herdam de Plano



Cliente --> Plano (associação) Cliente *-- Treino (composição) Treino *-- Exercicio (composição)



Fluxo principal

Cadastro do plano -> cadastro e validação do cliente -> associação do plano -> criação do treino -> inclusão dos exercícios -> cálculo polimórfico da mensalidade -> processamento do pagamento -> exibição do resultado.

4. Implementação e regras de negócio

A classe Cliente valida nome obrigatório, formato de e-mail e idade mínima de 15 anos. A associação só aceita planos ativos. Plano é abstrata e concentra os dados comuns; PlanoMensal devolve o valor base, enquanto PlanoPremium acrescenta a taxa de serviços. PagamentoService mantém a responsabilidade de pagamento separada das entidades.

Trecho central do polimorfismo

```
public interface Calculavel {
double calcularMensalidade();
}

public final class PlanoPremium extends Plano {
@Override
public double calcularMensalidade() {
return getValorBase() + taxaPremium;
}
}
```

Validações implementadas

- Nome obrigatório e sem conteúdo vazio.
- E-mail em formato válido e sem duplicidade no cadastro.
- Idade mínima de 15 anos.
- Identificadores e valores monetários positivos.
- Plano existente e ativo antes da associação ou pagamento.
- Treino e exercício não podem ser nulos; séries e repetições devem ser positivas.

Tratamento de erros

As exceções específicas são capturadas no fluxo de demonstração, produzindo mensagens compreensíveis. Isso impede que dados inválidos entrem no sistema e evita que uma falha esperada interrompa toda a execução.

5. Testes e evidências

O projeto foi compilado e testado com Java 21, Maven 3.9.9 e JUnit 5. Foram executados testes de unidade e regras de negócio, cobrindo entradas válidas, dados inválidos, polimorfismo, associação de plano, cadastro, duplicidade e pagamento.

Classe de teste	Testes	Falhas	Resultado
ClienteTest	5	0	APROVADO
PlanoTest	3	0	APROVADO
AcademiaServiceTest	2	0	APROVADO
PagamentoServiceTest	3	0	APROVADO
TOTAL	13	0	BUILD SUCCESS

Resumo da execução no console

```
PLANOS: Mensal = R$ 100,00 | Premium = R$ 250,00
CLIENTES: Kayky - Premium | Mateus - Mensal
TREINO: Supino reto 4x10 | Tríceps na polia 3x12
PAGAMENTOS: dois pagamentos APROVADOS
VALIDAÇÃO: e-mail inválido rejeitado e tratado
RESULTADO: Sistema executado com sucesso.
```

6. Como executar

Com Java 17 ou superior e Maven instalados, abra o terminal na pasta do projeto e use:

```
mvn clean test
mvn exec:java
```

Checklist da entrega

Item solicitado	Situação
Projeto funcional e organizado	**ATENDIDO**
Conceitos das Unidades 1, 2, 3 e 4	**ATENDIDO**
Cadastro de clientes e planos	**ATENDIDO**
Treinos e exercícios	**ATENDIDO**
Processamento de pagamentos	**ATENDIDO**
Validação e tratamento de exceções	**ATENDIDO**
Testes automatizados	**ATENDIDO**
README com instruções	**ATENDIDO**
Diagrama de classes	**ATENDIDO**
Evidências da execução	**ATENDIDO**

Conclusão

A implementação atende aos requisitos funcionais e aos conceitos de POO solicitados. A separação entre entidades, serviços e exceções deixa o código mais legível, testável e preparado para manutenção e evolução.